Energy Efficiency Center Industrial Walkthrough Checklist & Reference API

November 10, 2020

Prepared by

Zachary Thomas

CONTENTS

1 Introduction

This document describes all of the API endpoints for the EEC Walkthrough application. The document is organized by route name and each API endpoint includes a general description, the expected input, and the expected output.

This API uses cookies to keep track of the user who is making requests. This means that each of these endpoints will have access to all user information for the current user even if it is not requested in the body. This includes ID, username, first name, last name, email, and role. All endpoints also have the option to return an error code. This will have a status code between 400 and 500. In the case of a validation error an object with the property ”errors” will be returned, which is an array of objects that describe what was wrong with the request body. All other errors will return an object with the property ”error”, which is a string that describes the error.

To make handling responses from the API easier, all responses return a JSON object. To make it easier to tell what requests are being made to the API (as apposed to requests to get files), each API request starts with ”api”.

2 API Endpoints

2.1 Card Endpoints

POST api/cards

Creates a new card. Only editors and admins can use this endpoint.

- HeaderId: The ID of the header that this card belongs to.
- cardType: The type of card to create. Examples include: default card, internal card, thumbnail gallery, internal thumbnail gallery, and more.
- title: The title of the card.
- items: The items that will be included in the card.
  - cardId: The ID of the card that this item belongs to.
  - indentation: The level of indentation (0 is none, 3 is max).
  - iconType: The ID of the icon that is displayed at the start of the item.
  - contentText: The main body of text for the item.
  - contentUrl: The URL of the item. For graphic items this will be the URL of the image to display, for link items this will be the URL to direct the user to.
  - contentLabel: The bold title of the item, this is shown before the content text.
  - contentMode: A special setting that depends on the item in particular. For links this will be used to describe the type of link (internal, external, internal download, external download).
  - internal: Determines if the item should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
  - sourceId: The ID of the source that this item is referencing.

```
{
  headerId: 2,
  cardType: 0,
  title: "Best Practices",
  items: [
   {
```

```
    cardId: 1,
    indentation: 0,
    iconType: 16,
    contentText: "An informational page with analysis tools",
    contentUrl: "https://www.bpa.gov/EE/Sectors/Industrial/Pages/Co",
    contentLabel: "Bonneville Power Administration",
    contentMode: 0,
    internal: 0,
    sourceId: 2
   }
  ]
}
```

Listing 1. Example Request Body

- insertId: The ID of the newly created card.

```
{
  insertId: 10
}
```

Listing 2. Example Response

DELETE api/cards/:cardId/changes

Deletes an unpublished card / suggested changes to a card. Only editors and admins can use this endpoint.

- cardId: The ID of the card to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 3. Example Response

DELETE api/cards/:cardId

Deletes a card (including published cards). Only admins can use this endpoint.

- cardId: The ID of the card to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 4. Example Response

PATCH api/cards/:cardId

Updates a card. If the card is unpublished this will simply change the card. If the card is published then this will add a suggested edit to the card that will need to be published before it will replace the original. If the card is both published and has an edited version, then this will replace the edited version. Only an editor or an admins can use this endpoint.

- cardId: The ID of the card to update.
- cardType: The type of card to create. Examples include: default card, internal card, thumbnail gallery, internal thumbnail gallery, and more.
- title: The title of the card.
- items: The items that will be included in the card.
  - cardId: The ID of the card that this item belongs to.
  - indentation: The level of indentation (0 is none, 3 is max).
  - iconType: The ID of the icon that is displayed at the start of the item.
  - contentText: The main body of text for the item.
  - contentUrl: The URL of the item. For graphic items this will be the URL of the image to display, for link items this will be the URL to direct the user to.
  - contentLabel: The bold title of the item, this is shown before the content text.
  - contentMode: A special setting that depends on the item in particular. For links this will be used to describe the type of link (internal, external, internal download, external download).
  - internal: Determines if the item should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
  - sourceId: The ID of the source that this item is referencing.

```
{
  cardType: 0,
  title: "Best Practices",
  items: [
   {
     cardId: 1,
     indentation: 0,
     iconType: 16,
     contentText: "An informational page with analysis tools",
     contentUrl: "https://www.bpa.gov/EE/Sectors/Industrial/Pages/Co",
     contentLabel: "Bonneville Power Administration",
     contentMode: 0,
     internal: 0,
     sourceId: 2
   }
  ]
}
```

Listing 5. Example Request Body

- cardId: The ID of the current card.

```
{
  cardId: 1
}
```

Listing 6. Example Response

POST api/cards/:cardId/publish

Publishes a card. If the card is unpublished this will simply approve the card and show it publicly. If the card has a published version and an edit then the edit will replace the current published version while becoming approved. Only an admins can use this endpoint.

- cardId: The ID of the card to publish.

```
{
  cardId: 1
}
```

Listing 7. Example Response

POST api/cards/:cardId/unpublish

Unpublishes a card. If the card is published this will simply make the card unapproved and unviewable by the public. If the card has a published version and an edit then the edit will be replaced by the current published version while becoming unapproved. Only an admins can use this endpoint.

- cardId: The ID of the card to unpublish.

```
{
  cardId: 1
}
```

Listing 8. Example Response

PATCH api/cards/:cardId/move/:direction/:mode

Changes the selected card's position relative to other cards when displayed. If we are moving an unpublished card, it will move the card relative to all cards (excluding its published version). If we are moving a published card it will move relative to published cards (skipping over unpublished cards). Only editors or admins can move an unpublished card. Only admins can move a published card.

- cardId: The ID of the card to move.
- direction: A value of 1 moves the card up one row. A value of 0 moves the card down one row.
- mode: States what type of card we want to move. A value of 0 represents an unpublished card. A value of 1 represents a published card.

```
{
  cardId: 1
}
```

Listing 9. Example Response

GET api/cards/titles

Returns a list of suggested titles for cards. Editors can use these suggested titles to quickly name cards that are similar across pages. Any user can use this endpoint.

- titles: An array of strings that will be used as the suggested titles.

```
{
  titles: [
    "General Off Site Resource Links",
    "Best Practices",
    "Caveats"
  ]
}
```

Listing 10. Example Response

POST api/cards/titles

Replaces the current list of suggested card titles. Only admins can use this endpoint.

- titles: An array of strings that will be used as the new suggested titles.

```
{
  titles: [
    "Data Collection Guides",
    "Rules of Thumb",
    "Caveats"
  ]
}
```

Listing 11. Example Request Body

```
{
  titleCount: 3
}
```

Listing 12. Example Response

2.2 Category Endpoints

GET api/categories/all

Returns all of the categories and their respective pages. Users without an account, or external users will get all published categories and pages that are not internal. Internal users, editors, and admins will get all categories and pages.

- categoryId: The ID of the category.
- created: The date that the category was created.
- description: A general description of the category.
- internal: Whether the category is viewable by the public (value of 0) or just by internal users (value of 1).
- pluralName: The plural name of the category.
- singleName: The singular name of the category.
- userId: The ID of the user who created the category.
- pages: The list of pages that belong to the category.
  - approved: Whether this page is published and viewable in view mode.
  - description: The description of the page.
  - imageUrl: The URL of the image to be displayed at the top of the page.
  - internal: Whether the page is viewable by the public (value of 0) or just by internal users (value of 1).
  - name: The name of the page.
  - pageId: The ID of the page.
  - pageType: The category ID that the page belongs to.

```
{
  categories: [
    {
      categoryId: 5,
      created: "2020-07-18T03:11:32.000Z",
      description: "Standard recommended approaches for performing...",
      internal: 0,
      pluralName: "Assessments",
      singleName: "Assessment",
      userId: 51,
      pages: [
        approved: 1,
        description: "Cybersecurity is...",
        imageUrl: "/security.jpg"
        internal: 1,
        name: "Cybersecurity",
        pageId: 62,
        pageType: 5
      ]
    }
  ]
}
```

Listing 13. Example Response

GET api/categories/names

Returns the names and IDs of all categories. Users without an account, or external users will only have access to published categories that are not internal. Internal users, editors, and admins will have access to all categories.

- categoryId: The ID of the category.
- singleName: The singular name of the category.

```
{
  categories: [
    {
      categoryId: 5,
      singleName: "Assessment"
    }
  ]
}
```

Listing 14. Example Response

GET api/categories/:categoryId

Returns a single category and all of its pages. Users without an account, or external users will only have access to published categories and published pages that are not internal. Internal users, editors, and admins will have access to all categories and all pages.

- categoryId: The ID of the category.
- created: The date that the category was created.
- description: A general description of the category.
- internal: Whether the category is viewable by the public (value of 0) or just by internal users (value of 1).
- pluralName: The plural name of the category.
- singleName: The singular name of the category.
- userId: The ID of the user who created the category.
- pages: The list of pages that belong to the category.
  - approved: Whether this page is published and viewable in view mode.
  - description: The description of the page.
  - internal: Whether the page is viewable by the public (value of 0) or just by internal users (value of 1).
  - name: The name of the page.
  - pageId: The ID of the page.
  - pageType: The category ID that the page belongs to.

```
{
  categoryId: 5,
  created: "2020-07-18T03:11:32.000Z",
```

```
  description: "Standard recommended approaches for performing...",
  internal: 0,
  pluralName: "Assessments",
  singleName: "Assessment",
  userId: 51,
  pages: [
    approved: 1,
    description: "Cybersecurity is...",
    internal: 1,
    name: "Cybersecurity",
    pageId: 62,
    pageType: 5
  ]
}
```

Listing 15. Example Response

POST api/categories

Creates a new category. This endpoint can only be used by admins.

- description: A general description of the category.
- internal: Whether the category is viewable by the public (value of 0) or just by internal users (value of 1).
- pluralName: The plural name of the category.
- singleName: The singular name of the category.

```
{
  description: "Standard recommended approaches for performing...",
  internal: 0,
  pluralName: "Assessments",
  singleName: "Assessment",
}
```

Listing 16. Example Request Body

- insertId: The ID of the newly created category.

```
{
  insertId: 10
}
```

Listing 17. Example Response

PATCH api/categories/:categoryId

Updates a category. This endpoint can only be used by admins.

- categoryId: The ID of the category to be updated.
- description: A general description of the category.

- internal: Whether the category is viewable by the public (value of 0) or just by internal users (value of 1).
- pluralName: The plural name of the category.
- singleName: The singular name of the category.

```
{
  description: "Standard recommended approaches for performing...",
  internal: 0,
  pluralName: "Assessments",
  singleName: "Assessment",
}
```

Listing 18. Example Request Body

- categoryId: The ID of the category.

```
{
  categoryId: 10
}
```

Listing 19. Example Response

2.3 File Endpoints

POST api/files/single

Uploads a single file to the server. The file must use one of the following filename extensions: JPG, PNG, or GIF. If a file is successfully uploaded, then it is stored in a directory that is named after the user who uploaded the file. This allows users to be accountable for the content that they upload. Only editors or admins can upload files.

- fieldname: The form field name. For this endpoint it is always ”image”.
- originalname: The original file name.
- encoding: The file encoding.
- mimetype: The MIME type of the file. Similar to a filename extension.
- destination: The destination of the file on the server.
- filename: The newly generated filename.
- path: Similar to destination.
- size: The size of the file in bytes.

```
{
  fieldname: "image",
  originalname: "taco.gif",
  encoding: "7bit",
  mimetype: "image/gif",
  destination: "../fontend/public/uploads/user_42/",
  filename: "4fd7f3a50a0c63f368806aad8c36c3c5.gif",
  path: "..\\frontend\\public\\uploads\\user_42\\4fd7f3a50a0c63f368806aad8c36c3c5.gif",
  size: 487495
```

```
}
```

Listing 20. Example Request File

- url: The address that points to the newly uploaded file.

```
{
  url: "/uploads/user_42/4fd7f3a50a0c63f368806aad8c36c3c5.gif"
}
```

Listing 21. Example Response

POST api/files/bulk

Uploads any number of files to the server. The files must use one of the following filename extensions: JPG, PNG, or GIF. If a file is successfully uploaded, then it is stored in a directory that is named after the user who uploaded the file. This allows users to be accountable for the content that they upload. Only editors or admins can upload files.

- fieldname: The form field name. For this endpoint it is always ”images”.
- originalname: The original file name.
- encoding: The file encoding.
- mimetype: The MIME type of the file. Similar to a filename extension.
- destination: The destination of the file on the server.
- filename: The newly generated filename.
- path: Similar to destination.
- size: The size of the file in bytes.

```
[
  {
    fieldname: "images",
    originalname: "taco.gif",
    encoding: "7bit",
    mimetype: "image/gif",
    destination: "../fontend/public/uploads/user_42/",
    filename: "4fd7f3a50a0c63f368806aad8c36c3c5.gif",
    path: "..\\frontend\\public\\uploads\\user_42\\4fd7f3a50a0c63f368806aad8c36c3c5.gif",
    size: 487495
  },
  {
    fieldname: "images",
    originalname: "sushi.gif",
    encoding: "7bit",
    mimetype: "image/gif",
    destination: "../fontend/public/uploads/user_42/",
    filename: "76ecc168a8dfc7a39331f4dd23009e80.gif",
    path: "..\\frontend\\public\\uploads\\user_42\\76ecc168a8dfc7a39331f4dd23009e80.gif",
    size: 90112
  }
```

```
]
```

Listing 22. Example Request Files

- urls: An array of addresses that point to the newly uploaded files.

```
{
  urls: [
    "/uploads/user_42/4fd7f3a50a0c63f368806aad8c36c3c5.gif",
    "/uploads/user_42/76ecc168a8dfc7a39331f4dd23009e80.gif"
  ]
}
```

Listing 23. Example Response

POST api/files/directories

Returns information about the file upload directories that belong to each user. Only an admin can use this endpoint.

- sort: The value to sort results by (0 = username, 1 = user ID, 2 = number of files).
- order: The order in which to display results (0 = descending, 1 = ascending).
- cursor: The user ID associated with the first directory to be displayed. If you want to display results from the start of the list use null.

```
{
  sort: 0,
  order: 1,
  cursor: 11
}
```

Listing 24. Example Request Body

- directories: The list of directories.
  - name: The name of the user who owns the directory.
  - userId: The ID of the user who owns the directory.
  - fileCount: The number of uploaded files in the directory.
- nextCursor: The user ID associated with the next directory to be displayed. If there are no more directories then this value is null.

```
{
  directories: [
    {
      name: "Silverware",
      userId: 42,
      fileCount: 4
    }
  ],
  nextCursor: 11
```

```
}
```

Listing 25. Example Response

POST api/files/:userId

Returns information about the files a user has uploaded. Only the user who uploaded the files or an admin can use this endpoint.

- userId: The ID of the user who uploaded files.
- sort: The value to sort results by (0 = file name, 1 = source, 2 = used on website).
- order: The order in which to display results (0 = descending, 1 = ascending).
- cursor: The file name associated with the first file to be displayed. If you want to display results from the start of the list use null.

```
{
  sort: 0,
  order: 1,
  cursor: "null"
}
```

Listing 26. Example Request Body

- files: The list of files.
  - userId: The ID of the user who uploaded files.
  - url: A path pointing to the files location.
  - name: The name of the file.
  - used: Whether or not this file is referenced somewhere on the website.
  - source: The first valid source referenced along with the file on the website.
- nextCursor: The file name of the next file to be displayed. If there are no more files then this value is null.

```
{
  files: [
    {
      userId: 42,
      url: "/uploads/user_42/d0d105c1ba1fe16e613f69173867b797.jpg",
      name: "d0d105c1ba1fe16e613f69173867b797.jpg",
      used: "Yes",
      source: "Robotic Welding for Fabrication of MilSpec Hydra ..."
    }
  ],
  nextCursor: "b2da01c1ba1ca16b242a62145867b121.jpg"
}
```

Listing 27. Example Response

DELETE api/files/:userId/:fileName

Deletes a file. Only the user who uploaded the file or an admins can use this endpoint.

- userId: The ID of the user who uploaded the file.
- fileName: The name of the file.
- filesRemoved: The number of files removed.

```
{
  filesRemoved: 1
}
```

Listing 28. Example Response

2.4 Header Endpoints

POST api/headers

Creates a new header. Only editors and admins can use this endpoint.

- PageId: The ID of the page that this header belongs to.
- title: The title of the header.
- internal: Determines if the header should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.

```
{
  pageId: 15,
  title: "Wastewater Overview",
  internal: 0
}
```

Listing 29. Example Request Body

- insertId: The ID of the newly created header.

```
{
  insertId: 5
}
```

Listing 30. Example Response

DELETE api/headers/:headerId/changes

Deletes an unpublished header / suggested changes to a header. Only editors and admins can use this endpoint.

- headerId: The ID of the header to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 31. Example Response

DELETE api/headers/:headerId

Deletes a header (including published headers). Only admins can use this endpoint.

- headerId: The ID of the header to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 32. Example Response

PATCH api/headers/:headerId

Updates a header. If the header is unpublished this will simply change the header. If the header is published then this will add a suggested edit to the header that will need to be published before it will replace the original. If the header is both published and has an edited version, then this will replace the edited version. Only an editor or an admins can use this endpoint.

- headerId: The ID of the header to update.
- title: The title of the header.
- internal: Determines if the header should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.

```
{
  title: "Compressed Air Overview",
  internal: 0
}
```

Listing 33. Example Request Body

- headerId: The ID of the current header.

```
{
  headerId: 1
}
```

Listing 34. Example Response

POST api/headers/:headerId/publish

Publishes a header. If the header is unpublished this will simply approve the header and show it publicly. If the header has a published version and an edit then the edit will replace the current published version while becoming approved. Only an admins can use this endpoint.

- headerId: The ID of the header to publish.

```
{
  headerId: 1
}
```

Listing 35. Example Response

POST api/headers/:headerId/unpublish

Unpublishes a header. If the header is published this will simply make the header unapproved and unviewable by the public. If the header has a published version and an edit then the edit will be replaced by the current published version while becoming unapproved. Only an admins can use this endpoint.

- headerId: The ID of the header to unpublish.

```
{
  headerId: 1
}
```

Listing 36. Example Response

PATCH api/headers/:headerId/move/:direction/:mode

Changes the selected header's position relative to other headers when displayed. If we are moving an unpublished header, it will move the card relative to all cards (excluding its published version). If we are moving a published header it will move relative to published header (skipping over unpublished headers). Only editors or admins can move an unpublished header. Only admins can move a published header.

- headerId: The ID of the header to move.
- direction: A value of 1 moves the header up one row. A value of 0 moves the header down one row.
- mode: States what type of header we want to move. A value of 0 represents an unpublished header. A value of 1 represents a published header.

```
{
  headerId: 1
}
```

Listing 37. Example Response

2.5 Home Endpoints

GET api/home

Returns all of the data displayed on the homepage, with the exception of the sponsors. Any user can use this endpoint.

- mainHeader: The main header text for the homepage.
- secondaryHeader: The header for the card that describes the categories.
- sectionTitle: The title on the card that describes the categories.
- sectionFooter: The footer on the card that describes the categories.
- tidbitsHeader: The header for the card describing tidbits.
- tidbitsTitle: The title on the card that describes the tidbits.
- tidbitsFooter: The footer on the card that describes the tidbits.
- linksHeader: The header for the card describing links.
- linksTitlePrefix: The title on the card that is shown before the standard link icons are listed.
- linksTitlePostfixInternal: The title on the card that is shown before the link internal and external icons.
- linksTitlePostfixDownload: The title on the card that is shown before the link download icon.
- linksFooter: The footer on the card that describes the links.
- disclaimerHeader: The header of the disclaimer card.
- disclaimerText: The text on the disclaimer card.
- categories: The categories viewable by the current user.
  - categoryId: The ID of the category.
  - description: A general description of the category.
  - internal: Whether the category is viewable by the public (value of 0) or just by internal users (value of 1).
  - pluralName: The plural name of the category.
  - singleName: The singular name of the category.

```
{
  mainHeader: "Welcome",
  secondaryHeader: "The purpose of this guide...",
  sectionTitle: "This guide is broken down into sections:",
  sectionFooter: "",
  tidbitsHeader: "Each section...",
  tidbitsTitle: "These include...",
  tidbitsFooter: "Note: tidbit types can be...",
  linksHeader: "Each section also refrences...",
  linksTitlePrefix: "A preceding icon identifies...",
  linksTitlePostfixInternal: "A trailing icon...",
  linksTitlePostfixDownload: "A second trailing icon...",
  linksFooter: "",
  disclaimerHeader: "Disclaimer",
  disclaimerText: "The primary objective...",
  categories: [
    {
      categoryId: 5,
      description: "Standard recommended...",
      internal: 0,
```

```
      pluralName: "Assessments",
      singleName: "Assessment"
    }
  ]
}
```

Listing 38. Example Response

PATCH api/home

Updates the text that is displayed on the homepage. Only admins can use this endpoint.

- mainHeader: The main header text for the homepage.
- secondaryHeader: The header for the card that describes the categories.
- sectionTitle: The title on the card that describes the categories.
- sectionFooter: The footer on the card that describes the categories.
- tidbitsHeader: The header for the card describing tidbits.
- tidbitsTitle: The title on the card that describes the tidbits.
- tidbitsFooter: The footer on the card that describes the tidbits.
- linksHeader: The header for the card describing links.
- linksTitlePrefix: The title on the card that is shown before the standard link icons are listed.
- linksTitlePostfixInternal: The title on the card that is shown before the link internal and external icons.
- linksTitlePostfixDownload: The title on the card that is shown before the link download icon.
- linksFooter: The footer on the card that describes the links.
- disclaimerHeader: The header of the disclaimer card.
- disclaimerText: The text on the disclaimer card.

```
{
  mainHeader: "Welcome",
  secondaryHeader: "The purpose of this guide...",
  sectionTitle: "This guide is broken down into sections:",
  sectionFooter: "",
  tidbitsHeader: "Each section...",
  tidbitsTitle: "These include...",
  tidbitsFooter: "Note: tidbit types can be...",
  linksHeader: "Each section also refrences...",
  linksTitlePrefix: "A preceding icon identifies...",
  linksTitlePostfixInternal: "A trailing icon...",
  linksTitlePostfixDownload: "A second trailing icon...",
  linksFooter: "",
  disclaimerHeader: "Disclaimer",
  disclaimerText: "The primary objective..."
}
```

Listing 39. Example Request Body

- homePageUpdated: Generic value. Should always have a value of 1, showing that the home page was updated correctly.

```
{
  homePageUpdated: 1
}
```

Listing 40. Example Response

GET api/home/sponsors

Returns all of the sponsors that should be listed on the homepage. Any user can use this endpoint.

- sponsors: The sponsors for this application.
  - sponsorId: The ID of the sponsor.
  - name: The name of the sponsor.
  - title: A short description of the sponsor that doubles as a link.
  - websiteUrl: The URL for the sponsors website.
  - imageUrl: The URL for the sponsors image.
  - orderIndex: A value that determines the order that the sponsors are listed in. A lower value will appear before a larger value.

```
{
  sponsors: [
    {
      sponsorId: 5,
      name: "Industrial Assessment Center",
      title: "U.S. Department of Energy...",
      websiteUrl: "https://www.energy.gov/eere/amo/industrial-assessment-centers-iacs",
      imageUrl: "/images/IAC.png",
      orderIndex: 0
    }
  ]
}
```

Listing 41. Example Response

PATCH api/home/sponsors

Updates the list of sponsors that is displayed on the homepage. Only admins can use this endpoint.

- sponsors: The sponsors for this application.
  - sponsorId: The ID of the sponsor.
  - name: The name of the sponsor.
  - title: A short description of the sponsor that doubles as a link.

  - websiteUrl: The URL for the sponsors website.
  - imageUrl: The URL for the sponsors image.
  - orderIndex: A value that determines the order that the sponsors are listed in. A lower value will appear before a larger value.

```
{
  sponsors: [
    {
      sponsorId: 5,
      name: "Industrial Assessment Center",
      title: "U.S. Department of Energy...",
      websiteUrl: "https://www.energy.gov/eere/amo/industrial-assessment-centers-iacs",
      imageUrl: "/images/IAC.png",
      orderIndex: 0
    }
  ]
}
```

Listing 42. Example Request Body

- sponsorsUpdated: Generic value. Should always have a value of 1, showing that the sponsors were updated correctly.

```
{
  sponsorsUpdated: 1
}
```

Listing 43. Example Response

2.6 Icon Endpoints

GET api/icons/all

Returns a list of all icons that can be appended to items. Any user can use this endpoint.

- icons: All possible icons.
  - iconType: The ID of the icon.
  - groupIndex: The type of item this icon represents (disabled = 0, default item = 1, graphics item = 2, link item = 3).
  - typeKeyword: The name of the icon when it is being hovered over or selected in a menu.
  - typeName: The name of the actual Font Awesome icon.
  - color: The color of the icon. Represented by a hexadecimal color code.

```
{
  icons: [
    {
      iconType: 21,
```

```
      groupIndex: 3,
      typeKeyword: "Analysis Tool",
      typeName: "list",
      color: "#000000"
    }
  ]
}
```

Listing 44. Example Response

POST api/icons

Creates a new icon. Only an admin can use this endpoint.

- groupIndex: The type of item this icon represents (disabled = 0, default item = 1, graphics item = 2, link item = 3).
- typeKeyword: The name of the icon when it is being hovered over or selected in a menu.
- typeName: The name of the actual Font Awesome icon.
- color: The color of the icon. Represented by a hexadecimal color code.

```
{
  groupIndex: 3,
  typeKeyword: "Analysis Tool",
  typeName: "list",
  color: "#000000"
}
```

Listing 45. Example Request Body

- insertId: The ID of the newly created icon.

```
{
  insertId: 10
}
```

Listing 46. Example Response

PATCH api/icons/:iconId

Updates an icon. Only an admin can use this endpoint.

- iconId: The ID of the icon.
- groupIndex: The type of item this icon represents (disabled = 0, default item = 1, graphics item = 2, link item = 3).
- typeKeyword: The name of the icon when it is being hovered over or selected in a menu.
- typeName: The name of the actual Font Awesome icon.
- color: The color of the icon. Represented by a hexadecimal color code.

```
{
  groupIndex: 3,
  typeKeyword: "Analysis Tool",
  typeName: "list",
  color: "#000000"
}
```

Listing 47. Example Request Body

- iconId: The ID of the icon.

```
{
  iconId: 10
}
```

Listing 48. Example Response

2.7 Link Endpoints

POST api/links/all

Returns a list of all of the published link items. Only admins can use this endpoint.

- onlyDead: This value determines if we want to view all published links or just published links that have been marked invalid (0 = all links, 1 = invalid links only)
- sort: The value to sort results by (0 = last confirmed valid time, 1 = title, 2 = url).
- order: The order in which to display results (0 = descending, 1 = ascending).
- cursorPrimary: This cursor should contain the value that we are sorting by of the link that we want to start our results with. This cursor is used to allow infinite scrolling, while only loading a small number of page results at a time. If this is a new search then this value should be null.
- cursorSecondary: This cursor should contain the item ID of the link that we want to start our results with. This cursor is used in instances where the primary cursor value is ambiguous. If this is a new search then this value should be null.

```
{
  onlyDead: 0,
  sort: 0,
  order: 1,
  cursorPrimary: "null"
  cursorSecondary: "null"
}
```

Listing 49. Example Request Body

- links: All links.
  - itemId: The ID of the link item.
  - time: The time the link was last confirmed valid. This value is null if the link is currently invalid.

  - unixTime: The time the link was last confirmed valid. This value is represented as the number of seconds that have elapsed since the Unix epoch. This value is null if the link is currently invalid.
  - title: The text that the link is embedded in.
  - url: The URL for the link.

- nextCursor: An object that contains the information needed to continue getting the remaining results.
  - cursorPrimary: This cursor should contain the value that we are sorting by of the link that should appear next in the search results. If this is null, then there are no more links.
  - cursorSecondary: This cursor should contain the item ID of the link that should appear next in the search results. If this is null, then there are no more links.

```
{
  links: [
    {
      itemId: 384,
      time: "2020-06-29T20:55:45.000Z",
      unixTime: 1593464145,
      title: "U.S. Department of Energy analysis tool",
      url: "https://www.energy.gov/eere/amo/measur"
    }
  ],
  nextCursor: {
    primary: "1601157152",
    secondary: "6287"
  }
}
```

Listing 50. Example Response

PATCH api/links/:linkId

Changes the URL of a link item, and updates the last confirmed valid date to the current time. Only admins can use this endpoint.

- linkId: The ID of the link item.
- url: The new URL for the link item.

```
{
  url: "https://www.bpa.gov/EE/Sectors/Industrial/Pages/Compressed-Air.aspx"
}
```

Listing 51. Example Request Body

- linkId: The ID of the link item.

```
{
  linkId: 10
}
```

Listing 52. Example Response

PATCH api/links/:linkId/timestamp

Updates a link item's confirmed valid date to the current time. Only editors or admins can use this endpoint.

- linkId: The ID of the link item.
- timestamp: The time that the link was confirmed valid.

```
{
  timestamp: "2020-06-29T20:55:45.00Z"
}
```

Listing 53. Example Response

2.8 Notification Endpoints

GET api/notifications

Returns a list of all notifications that belong to the current user. Any user can use this endpoint.

- notifications: All notifications that belong to the current user.
  - notificationId: The ID of the notification.
  - requestId: If this notification is related to a request, then this will be the ID of the request.
  - userId: The ID of the user who the notification belongs to.
  - text: The message that is displayed to the user.
  - type: A number representing the type of notification. This value can be used to delete groups of notifications when some event happens.

```
{
  notifications: [
    {
      notificationId: 15,
      requestId: 25,
      userId: 42,
      text: "The request 'New Request' is awaiting an orange review",
      type: 1
    }
  ]
}
```

Listing 54. Example Response

DELETE api/notifications/:notificationId

Deletes a notification. Only the user who the notification belongs to, can use this endpoint.

- notificationId: The ID of the notification to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 55. Example Response

2.9 Page Endpoints

GET api/pages/:pageId/all

Returns all of the page, header, card, and item data for a single page. Any user can use this endpoint. Only internal users will be allowed to get info about private or unpublished pages, headers, cards, and items.

- pageId: The ID of the page.
- pageType: The ID of the category that this page belongs to.
- name: The name of the page.
- title: The title of the initial card that shows at the top of the page.
- description: Text that describes the general subject of the page.
- imageUrl: The URL for the image shown at the top of the page.
- created: The time that the page was created, last updated, or last published (whichever was most recent).
- userId: The ID of the user who created or last updated the page (whichever was most recent).
- approved: Lets us know if the page is published (1) or unpublished (0).
- internal: Determines if the page should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
- tempPageId: If the page is published and there is an edited version the page, then this is the same as the pageId, otherwise null.
- tempPageType: If the page is published and there is an edited version the page, then this is the same as the pageType, except in relation to the unpublished version, otherwise null.
- tempTitle: If the page is published and there is an edited version the page, then this is the same as the title, except in relation to the unpublished version, otherwise null.
- tempUserId: If the page is published and there is an edited version the page, then this is the same as the userId, except in relation to the unpublished version, otherwise null.
- tempImageUrl: If the page is published and there is an edited version of the page, then this is similar to imageUrl, except in relation to the unpublished version, otherwise null.
- sources: All of the sources that are referenced on this page.
  - pageId: The page ID of the page that this source belongs to.
  - sourceId: The ID of this source.
  - text: The citation text that describes this source (IEEE style).

  - url: An optional URL that links to a web page related to the source.

- headers: All of the headers that are on this page.
  - headerId: The ID of the header.
  - pageId: The ID of the page that this header belongs to.
  - orderIndex: The order that the headers should be displayed in. A lower value means the header is displayed before a header with a higher value.
  - title: The title of the header.
  - internal: Determines if the header should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
  - created: The time that the header was created, last updated, or last published (whichever was most recent).
  - userId: The ID of the user who created or last updated the header (whichever was most recent).
  - approved: Lets us know if the page is published (1) or unpublished (0).
  - tempHeaderId: If the header is published and there is an edited version of the header, then this is the same as headerId, otherwise null.
  - tempCreated: If the header is published and there is an edited version of the header, then this is the time that the edited version was created or last updated, otherwise null.
  - tempInternal: If the header is published and there is an edited version of the header, then this is similar to internal, except in relation to the unpublished version, otherwise null.
  - tempOrderIndex: If the header is published and there is an edited version of the header, then this is similar to orderIndex, except in relation to the unpublished version, otherwise null.
  - tempTitle: If the header is published and there is an edited version of the header, then this is similar to title, except in relation to the unpublished version, otherwise null.
  - tempUserId: If the header is published and there is an edited version of the header, then this is similar to userId, except in relation to the unpublished version, otherwise null.
  - cards: The cards that belong to this header.
    * cardId: The ID of the card.
    * HeaderId: The ID of the header that this card belongs to.
    * cardType: The type of card to create. Examples include: default card, internal card, thumbnail gallery, internal thumbnail gallery, and more.
    * title: The title of the card.
    * created: The time that the card was created, last updated, or last published (whichever was most recent).
    * userId: The ID of the user who created or last updated the card (whichever was most recent).
    * approved: Lets us know if the card is published (1) or unpublished (0).
    * orderIndex: The order that the cards should be displayed in. A lower value means the card is displayed before a card with a higher value.
    * tempCardId: If the card is published and there is an edited version of the card, then this is the same as cardId, otherwise null.

- tempCardType: If the card is published and there is an edited version of the card, then this is similar to cardType, except in relation to the unpublished version, otherwise null.
- tempCreated: If the card is published and there is an edited version of the card, then this is the time that the edited version was created or last updated, otherwise null.
- tempOrderIndex: If the card is published and there is an edited version of the card, then this is the similar to orderIndex, except in relation to the unpublished version, otherwise null.
- tempTitle: If the card is published and there is an edited version of the card, then this is the similar to title, except in relation to the unpublished version, otherwise null.
- tempUserId: If the card is published and there is an edited version of the card, then this is the similar to userId, except in relation to the unpublished version, otherwise null.
- cards: The cards that belong to this header.
- items / tempItems: The items that belong to this card. "items" are always approved (1), tempItems are always unapproved (0).
  - itemId: The ID of the item.
  - cardId: The ID of the card that this item belongs to.
  - indentation: The level of indentation (0 is none, 3 is max).
  - iconType: The ID of the icon that is displayed at the start of the item.
  - contentText: The main body of text for the item.
  - contentUrl: The URL of the item. For graphic items this will be the URL of the image to display, for link items this will be the URL to direct the user to.
  - contentLabel: The bold title of the item, this is shown before the content text.
  - contentMode: A special setting that depends on the item in particular. For links this will be used to describe the type of link (internal, external, internal download, external download).
  - internal: Determines if the item belongs to a published (1) or unpublished (0) version of the card.
  - iconType: The ID of the icon to use with this item.
  - typeName: The Font Awesome name of the icon that is used with this item.
  - typeKeyWord: The name displayed when a user hovers over the icon.
  - color: The hex color code to use for the icon.
  - sourceId: The ID of the source that this item is referencing.


```
{
  pageId: 2,
  pageType: 2,
  name: "Compressed Air",
  title: "Compressed air is a common utilitiy...",
  description: "Compressed air has been a key...",
  imageUrl: "/images/air.png",
  created: "2020-07-23T10:01:49.000Z",
  userId: 42,
  approved: 1,
  internal: 0,
  tempCreated: null,
  tempDescription: null,
```

```
tempImageUrl: null,
tempInternal: null,
tempImageUrl: null,
tempName: null,
tempPageId: null
sources: [
  {
    sourceId: 1,
    pageId: 2,
    text: "S. Cottrell, the study skills handbook...",
    url: ""
  }
],
headers: [
  {
    headerId: 1,
    pageId: 2,
    approved: 1,
    internal: 0,
    orderIndex: 1,
    created: "2020-07-01T18:19:56.000Z",
    title: "Compressed Air Overview",
    userId: 51,
    tempCreated: null,
    tempHeaderId: null,
    tempInternal: null,
    tempOrderIndex: null,
    tempTitle: null,
    tempUserId: null
    cards: [
      {
        cardId: 9,
        cardType: 0,
        approved: 1,
        created: "2020-06-02T20:58:31.000Z",
        headerId: 1,
        orderIndex: 3,
        title: "Pros",
        userId: 42,
        tempCardId: 9,
        tempCardType: 0,
        tempCreated: "2020-09-03T23:15:22.000Z",
        tempOrderIndex: 3,
        tempTitle: "Pros",
        tempUserId: 58
        items [
          {
            approved: 1,
            itemId: 25,
            cardId: 9,
```

```
              color: "#000000",
              contentLabel: "",
              contentMode: 0,
              contentText: "Versatile. Offers compact energy density.",
              contentUrl: "",
              created: "2020-06-02T22:38:04.000Z",
              internal: 0
              indentation: 0,
              iconType: 1,
              typeName: "plus",
              typeKeyword: "Pros",
              sourceId: 0
            }
          ],
          tempItems: [
            }
              approved: 0,
              itemId: 25,
              cardId: 9,
              color: "#000000",
              contentLabel: "",
              contentMode: 0,
              contentText: "Versatile. Energy can be stored in a compact way.",
              contentUrl: "",
              created: "2020-09-03T23:15:49.000Z",
              internal: 0
              indentation: 0,
              iconType: 1,
              typeName: "plus",
              typeKeyword: "Pros",
              sourceId: 0
            }
          ]
        }
      ]
    }
  ]
}
```

Listing 56. Example Response

GET api/pages/search/:text/:cursorPrimary/:cursorSecondary

Returns a limited list of pages that match the search text and that only includes pages that are at or beyond the cursor. Any user can use this endpoint, however internal or unpublished pages will only be returned for internal users.

- text: The partial or full text that is used to find a match. Partial matches are allowed. For example: searching for ”press” would be able to return ”Compressed Air” since ”press” is contained with in ”Compressed Air”.

- cursorPrimary: This cursor should contain the name of the page you want to start your search at (alphabetical order). This cursor is used to allow infinite scrolling, while only loading a small number of page results at a time. If this is a new search then this value should be null.
- cursorSecondary: This cursor should contain the page ID of the page you want to start your search at. This cursor is used in instances where the primary cursor value is ambiguous. If this is a new search then this value should be null.
- nextCursor: An object that contains the information needed to continue searching the rest of the pages (the primary and secondary cursor).
- pages: The pages returned from the search.
    - pageId: The ID of the page.
    - created: The time that the page was created, last updated, or last published (whichever was most recent).
    - description: Text that describes the general subject of the page.
    - imageUrl: The URL for the image shown at the top of the page.
    - internal: Determines if the page should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
    - approved: Lets us know if the page is published (1) or unpublished (0).
    - name: The name of the page.
    - pageType: The ID of the category that this page belongs to.
    - title: The title of the initial card that shows at the top of the page.
    - url: The URL needed to access the page.
    - userId: The ID of the user who created or last updated the page (whichever was most recent).

```
{
  nextCursor: {
    primary: null,
    secondary: null
  },
  pages: [
    pageId: 46,
    created: "2020-07-02T19:39:56.000Z",
    description: "Steam energy offered a great...",
    imageUrl: "https://live.staticflickr.com/65535/...",
    internal: 0,
    approved: 1,
    name: "Boilders and Steam",
    pageType: 2,
    title: "Boilers and Steam Systems are found in a large...",
    url: "/wiki/search-results/46",
    userId: 51
  ]
}
```

Listing 57. Example Response

POST api/pages

Creates a new page. Only editors and admins can use this endpoint.

- pageType: The ID of the category that this page belongs to.
- name: The name of the page.
- title: The title of the initial card that shows at the top of the page.
- description: Text that describes the general subject of the page.
- imageUrl: The URL for the image shown at the top of the page.
- internal: Determines if the page should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.

```
{
  pageType: 1,
  name: "Wastewater Treatment",
  title: "Municipalities and industry need...",
  description: "Wastewater treatment systems can...",
  imageUrl: "https://live.staticflickr.com/65535/...",
  internal: 0
}
```

Listing 58. Example Request Body

- insertId: The ID of the newly created page.

```
{
  insertId: 50
}
```

Listing 59. Example Response

DELETE api/pages/:pageId/changes

Deletes an unpublished page / suggested changes to a page. Only editors and admins can use this endpoint.

- pageId: The ID of the page to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 60. Example Response

DELETE api/pages/:pageId

Deletes a page (including published pages). Only admins can use this endpoint.

- pageId: The ID of the page to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 61. Example Response

PATCH api/pages/:pageId

Updates a page. If the page is unpublished this will simply change the page. If the page is published then this will add a suggested edit to the page that will need to be published before it will replace the original. If the page is both published and has an edited version, then this will replace the edited version. Only an editor or an admins can use this endpoint.

- pageId: The ID of the page to update.
- pageType: The ID of the category that this page belongs to.
- name: The name of the page.
- title: The title of the initial card that shows at the top of the page.
- description: Text that describes the general subject of the page.
- imageUrl: The URL for the image shown at the top of the page.
- internal: Determines if the page should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.

```
{
  pageType: 1,
  name: "Wastewater Treatment",
  title: "Municipalities and industry need...",
  description: "Wastewater treatment systems can...",
  imageUrl: "https://live.staticflickr.com/65535/...",
  internal: 0
}
```

Listing 62. Example Request Body

- pageId: The ID of the current page.

```
{
  pageId: 1
}
```

Listing 63. Example Response

POST api/pages/:pageId/publish

Publishes a page. If the page is unpublished this will simply approve the page and show it publicly. If the page has a

published version and an edit then the edit will replace the current published version while becoming approved. Only an admins can use this endpoint.

- pageId: The ID of the page to publish.

```
{
  pageId: 1
}
```

Listing 64. Example Response

POST api/pages/:pageId/unpublish

Unpublishes a page. If the page is published this will simply make the page unapproved and unviewable by the public. If the page has a published version and an edit then the edit will be replaced by the current published version while becoming unapproved. Only an admins can use this endpoint.

- pageId: The ID of the page to unpublish.

```
{
  pageId: 1
}
```

Listing 65. Example Response

GET api/pages/report/:start/:end/:offset/:condense

Returns a report of all of the published content within the date range. Only editors and admins can use this endpoint.

- start: The start date of the report (ex: 2020-09-01).
- end: The end date of the report (ex: 2020-09-06).
- offset: The minute offset of the time zone that the request is being made for, in relation to UTC (ex: 420 for PDT).
- condense: Whether to condense changes for an individual object into a single entry in the report, or to include every change as its own entry in the report.
- pages: Pages that were published or deleted within the date range.
  - historyId: The ID of this history record.
  - removed: If this history record is noting the removal of the page, then this value will be 1. If this history record is signifying that the page was published, then the value will be 0.
  - oldVersion: The previous version of this page. It has the following properties: created, description, historyId, imageUrl, internal, name, pageId, pageType, removed, and title.
  - pageId: The ID of the page.
  - pageType: The ID of the category that this page belongs to.
  - name: The name of the page.
  - title: The title of the initial card that shows at the top of the page.

  - description: Text that describes the general subject of the page.
  - imageUrl: The URL for the image shown at the top of the page.
  - created: The time that the page was created, last updated, or last published (whichever was most recent).
  - userId: The ID of the user who created or last updated the page (whichever was most recent).
  - approved: Lets us know if the page is published (1) or unpublished (0).
  - internal: Determines if the page should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
- headers: Headers that were published or deleted within the date range.
  - historyId: The ID of this history record.
  - removed: If this history record is noting the removal of the header, then this value will be 1. If this history record is signifying that the header was published, then the value will be 0.
  - oldVersion: The previous version of this header. It has the following properties: created, headerId, historyId, internal, pageId, removed, and title.
  - headerId: The ID of the header.
  - pageId: The ID of the page that this header belongs to.
  - orderIndex: The order that the headers should be displayed in. A lower value means the header is displayed before a header with a higher value.
  - title: The title of the header.
  - internal: Determines if the header should be viewable by the public, or just internal users. A value of 1 is only viewable by internal users, while a value of 0 is viewable by everyone.
  - created: The time that the header was created, last updated, or last published (whichever was most recent).
  - userId: The ID of the user who created or last updated the header (whichever was most recent).
  - approved: Lets us know if the page is published (1) or unpublished (0).
- cards: Cards that were published or deleted within the date range.
  - historyId: The ID of this history record.
  - removed: If this history record is noting the removal of the page, then this value will be 1. If this history record is signifying that the page was published, then the value will be 0.
  - oldVersion: The previous version of this card. It has the following properties: cardId, cardType, created, headerId, historyId, items, removed, and title.
  - cardId: The ID of the card.
  - HeaderId: The ID of the header that this card belongs to.
  - cardType: The type of card to create. Examples include: default card, internal card, thumbnail gallery, internal thumbnail gallery, and more.
  - title: The title of the card.
  - created: The time that the card was created, last updated, or last published (whichever was most recent).
  - userId: The ID of the user who created or last updated the card (whichever was most recent).
  - approved: Lets us know if the card is published (1) or unpublished (0).
  - orderIndex: The order that the cards should be displayed in. A lower value means the card is displayed before a card with a higher value.

- items: The items that belong to this card.
  - itemId: The ID of the item.
  - cardId: The ID of the card that this item belongs to.
  - indentation: The level of indentation (0 is none, 3 is max).
  - iconType: The ID of the icon that is displayed at the start of the item.
  - contentText: The main body of text for the item.
  - contentUrl: The URL of the item. For graphic items this will be the URL of the image to display, for link items this will be the URL to direct the user to.
  - contentLabel: The bold title of the item, this is shown before the content text.
  - contentMode: A special setting that depends on the item in particular. For links this will be used to describe the type of link (internal, external, internal download, external download).
  - internal: Determines if the item belongs to a published (1) or unpublished (0) version of the card.
  - iconType: The ID of the icon to use with this item.
  - typeName: The Font Awesome name of the icon that is used with this item.
  - typeKeyWord: The name displayed when a user hovers over the icon.
  - color: The hex color code to use for the icon.
  - sourceId: The ID of the source that this item is referencing.


```
{
  pages: [
    {
      categoryName: "Technologies",
      created: "2020-09-03T10:12:55.000Z",
      description: "Lorem ipsum...",
      historyId: 12,
      imageUrl: "https://placekitten.com/600/500",
      internal: 1,
      name: "New Page",
      pageId: 59,
      pageType: 2,
      removed: 0,
      sortType: 0,
      title: "A page for testing",
      oldVersion: {...}
    }
  ],
  headers: [
    {
      categoryName: "Industries",
      created: "2020-08-20T00:22:00.000Z",
      description: "Wastewater treatment systems can address a multitude...",
      historyId: 14,
      imageUrl: "https://live.satsicflickr...",
      internal: 0,
      name: "Wastewater Treatment",
      pageId: 50,
```

```
      pageType: 1,
      removed: 0,
      sortType: 0,
      title: "Municipalities and industry...",
      oldVersion: {...}
    }
  ],
  cards: [
    {
      cardId: 13,
      cardType: 0,
      categoryName: "Technologies",
      created: "2020-05-23T22:20:20.000Z",
      headerId: 1,
      headerName: "Compressed Air Overview",
      historyId: 4,
      pageId: 2,
      pageName: "Compressed Air",
      pageType: 2,
      removed: 0,
      sortType: 2,
      title: "Cons",
      oldVersion: {...},
      items: [
        {
          itemId: 32,
          cardId: 13,
          color: "#000000",
          contentLabel: "",
          contentMode: 0,
          contentText: "Extremely energy...",
          contentUrl: "",
          created: "2020-05-23T22:52:18.000Z",
          iconType: 2,
          indentation: 0,
          internal: 0,
          itemId: 32,
          orderIndex: 1,
          typeKeyword: "Cons",
          typeName: "minus"
        }
      ]
    }
  ]
}
```

Listing 66. Example Response

2.10 Request Endpoints

POST api/requests/status/:status

Returns a list of all publish requests that match the given status. Only editors and admins can use this endpoint.

- status: The status of the requests to return. A status of 0 returns all of the requests that have a status other than 4 (the status of closed requests).
- sort: The value to sort results by (0 = time created, 1 = title, 2 = username of the user who created the request, 3 = request status).
- order: The order in which to display results (0 = descending, 1 = ascending).
- cursorPrimary: This cursor should contain the value that we are sorting by of the request that we want to start our results with. This cursor is used to allow infinite scrolling, while only loading a small number of page results at a time. If this is a new search then this value should be null.
- cursorSecondary: This cursor should contain the request ID of the request that we want to start our results with. This cursor is used in instances where the primary cursor value is ambiguous. If this is a new search then this value should be null.

```
{
  sort: 0,
  order: 1,
  cursorPrimary: "null"
  cursorSecondary: "null"
}
```

Listing 67. Example Request Body

- requests: All publish requests that are awaiting approval.
  - requestId: The ID of the publish request.
  - status: The status of the publish request.
  - title: The title of the publish request.
  - description: The description of the request. This will often include more information about the request and why it is needed.
  - userId: The ID of the user who has opened the publish request.
  - userName: The username of the user who has opened the publish request.
  - created: The date that the publish request was created.
- nextCursor: An object that contains the information needed to continue getting the remaining results.
  - cursorPrimary: This cursor should contain the value that we are sorting by of the requests that should appear next in the search results. If this is null, then there are no more requests.
  - cursorSecondary: This cursor should contain the request ID of the requests that should appear next in the search results. If this is null, then there are no more requests.

```
{
  requests: [
    {
```

```
      requestId: 2,
      status: 1,
      title: "New Request",
      description: "This request includes a few pages that I want approved",
      userId: 58,
      userName: "testUser",
      created: "2020-09-04T20:55:06.000Z",
    }
  ],
  nextCursor: {
    primary: "3",
    secondary: "325"
  }
}
```

Listing 68. Example Response

GET api/requests/:requestId

Returns all the data related to a publish request. Only editors and admins can use this endpoint.

- requestId: The ID of the publish request.
- status: The status of the publish request.
- title: The title of the publish request.
- description: The description of the request. This will often include more information about the request and why it is needed.
- userId: The ID of the user who has opened the publish request.
- userName: The username of the user who has opened the publish request.
- created: The date that the publish request was created.
- comments: Comments left on the request by users.
  - commentId: The ID of the comment.
  - requestId: The ID of the publish request that this comment is on.
  - comment: The actual text of the comment.
  - review: The type of comment (0 = comment, 1 = suggest change, 2 = approval).
  - userId: The ID of the user who made the comment.
  - userName: The name of the user who made the comment.
- objects: The objects (pages, headers, and/or cards) that are being selected as part of the publish request.
  - objectType: The type of object that this is (1 = page, 2 = header, 3 = card).
  - The properties that the object will have is based on the object type. Pages will have the following additional properties: approved, categoryName, created, description, imageUrl, internal, name, pageId, pageType, title, and userId. Headers will have the following additional properties: approved, categoryName, created, headerId, internal, orderIndex, pageId, pageName, pageType, title, and userId. Cards will have the fol-

lowing additional properties: approved, cardId, cardType, categoryName, created, headerId, headerName, items, objectType, orderIndex, pageId, pageName, title, and userId.

```
{
  requestId: 2,
  status: 1,
  title: "New Request",
  description: "This request includes a few pages that I want approved",
  userId: 58,
  userName: "testUser",
  created: "2020-09-04T20:55:06.000Z",
  comments: [
    {
      commentId: 2,
      requestId: 2,
      comment: "Looks good.",
      review: 2,
      userId: 42,
      username: "Silverware"
    }
  ],
  objects: [
    {
      objectType: 1,
      pageId: 48,
      pageType: 2,
      approved: 0,
      categoryName: "Technologies",
      created: "2020-07-02T22:10:36.000Z",
      description: "Vapor compression is...",
      imageUrl: "https:///live.staticflickr.com/65535/...",
      internal: 0,
      name: "Refrigeration",
      objectType: 1,
      pageId: 48,
      pageType: 2,
      title: "Refrigeration technology...",
      userId: 51
    }
  ]
}
```

Listing 69. Example Response

POST api/requests/selections

Allows a user to use any number of object's type and ID to get more detailed information about them. Only editors and admins can use this endpoint.

- objects: The objects that the user wants more information about.

  - objectType: The type of the object (1 = page, 2 = header, 3 = card).
  - objectId: The ID of the object.

```
{
  objects: [
    {
      objectType: 1,
      objectId: 50
    }
  ]
}
```

Listing 70. Example Request Body

- objects: The objects that the user requested more information about.
  - objectType: The name of the object type.
  - key: A key that is the objects ID with the first letter of its object type appended to it.
  - objectName: The name or title of the object.

```
{
  objects: [
    {
      objectType: "Header",
      key: "32H",
      objectName: "Boilers and Steam Opportunities to Consider"
    }
  ]
}
```

Listing 71. Example Response

POST api/requests/

Creates a new publish request. Only editors and admins can use this endpoint.

- description: The description of the request. This will often include more information about the request and why it is needed.
- title: The title of the publish request.
- objects: The objects that are part of the publish request.
  - objectType: The type of the object (1 = page, 2 = header, 3 = card).
  - objectId: The ID of the object.

```
{
  description: "This new request will add 3 pages, and 2 headers that are ready",
  title: "Tim's Request",
  objects: [
    {
```

```
      objectType: 3,
      objectId: 168
    }
  ]
}
```

Listing 72. Example Request Body

- insertId: The ID of the new publish request.

```
{
  insertId: 4
}
```

Listing 73. Example Response

DELETE api/requests/:requestId

Closes a publish request. This doesn't actually delete the request, but instead gives it the closed status (4). When a request is closed it can't be interacted with, aside from viewing it. Only the creator of the request and admins can use this endpoint.

- requestId: The ID of the publish request to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 74. Example Response

POST api/requests/comment/:requestId

Creates a new comment on a publish request. Only editors and admins can use this endpoint.

- requestId: The ID of the publish request to add a comment to.
- comment: The actual text of the comment.
- status: The type of comment (0 = comment, 1 = suggest change, 2 = approval).
- targetId: A string that describes the specific object being commented on. If this is a general comment than targetId is ”0”. If this is a page it starts with a ”P”, header starts with an ”H”, card starts with a ”C”, and then the objects ID.

```
{
  comment: "Did you want to do anything else with the image title?",
  status: "0",
  targetID: "C121"
}
```

Listing 75. Example Request Body

- insertId: The ID of the new comment.

```
{
  insertId: 4
}
```

Listing 76. Example Response

PATCH api/requests/comment/:commentId

Updates a comment that exists on a publish request. Only the creator of the comment can use this endpoint.

- commentId: The ID of the comment to update.
- commentText: The new text to use for the comment.

```
{
  commentText: "This is an edited comment",
}
```

Listing 77. Example Request Body

- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 78. Example Response

DELETE api/requests/comment/:commentId

Deletes a comment that exists on a publish request. Only the creator of the comment can use this endpoint.

- requestId: The ID of the comment to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 79. Example Response

POST api/requests/accept/:requestId

Accepts the publish request. This causes all of the objects that are part of the publish request to become published. The request is then closed. This doesn't delete the request, but instead gives it the closed status (4). When a request is closed it can't be interacted with, aside from viewing it. Only an admins can use this endpoint.

- requestId: The ID of the publish request to accept.
- objectsApproved: The number of objects that were published by the request.

```
{
  objectsApproved: 5
}
```

Listing 80. Example Response

2.11 Source Endpoints

GET api/sources/page/:pageId

Returns all of the sources on a page (even the ones not being used by an item). Only editors and admins can use this endpoint.

- pageId: The ID of the page to get the sources from.
- sources: All of the sources that belong to the page.
  - pageId: The page ID of the page that this source belongs to.
  - sourceId: The ID of this source.
  - text: The citation text that describes this source (IEEE style). This text may include HTML tags.

```
{
  sources: [
    {
      pageId: 2,
      sourceId: 1,
      text: "S. Cottrell, The study skills handbook..."
    }
  ]
}
```

Listing 81. Example Response

POST api/sources/all/page/:pageId

Creates a list of sources for a page. Since this new list of sources replaces the previous one, it can be used as a way to create, edit, and delete sources. Only admins can use this endpoint.

- pageId: The ID of the page to post the source list to.
- sources: All of the sources to add to the page.
  - text: The citation text that describes this source (IEEE style). This text may include HTML tags.

```
{
  sources: [
    {
      text: "S. Cottrell, The study skills handbook..."
    }
  ]
```

```
}
```

Listing 82. Example Request Body

- sourcesApproved: The number of sources that were successfully added from the new source list.

```
{
  sourcesApproved: 1
}
```

Listing 83. Example Response

POST api/sources/page/:pageId

Adds a single source to a page. Only editors and admins can use this endpoint.

- pageId: The ID of the page to add the source to.
- text: The citation text that describes this source (IEEE style).

```
{
  text: "S. Cottrell, The study skills handbook..."
}
```

Listing 84. Example Request Body

- insertId: The ID of the new source.

```
{
  insertId: 1
}
```

Listing 85. Example Response

2.12 User Endpoints

GET api/users/:userId

Returns basic information about a user. Any logged in user can use this endpoint.

- userId: The ID of the user to get information about.
- created: The date that the user account was created.
- email. The user's email address.
- firstName: The user's first name.
- lastName: The user's last name.
- role: The user's role (0 = not logged in, 1 = external user, 2 = internal user, 3 = editor, 4 = admin).
- userId: The ID of the user.
- username: The username of the user.

```
{
  created: "2020-05-14T20:39:15.000Z",
  email: "thomasza@oregonstate.edu",
  firstName: "Zachary",
  lastName: "Thomas",
  role: 4,
  userId: 42,
  username: "Silverware"
}
```

Listing 86. Example Response

POST api/users/login

Attempts to login the user. If the user's username and password are correct, then they will be given cookies that will confirm their identity for use with the other endpoints. Any user can use this endpoint.

- username: The username of the account that the user wants to login to.
- password: A password that should match the username's password.

```
{
  username: "newUser",
  password: "hunter2"
}
```

Listing 87. Example Request Body

- email: The user's email address.
- firstName: The user's first name.
- lastName: The user's last name.
- role: The user's role (0 = not logged in, 1 = external user, 2 = internal user, 3 = editor, 4 = admin).
- userId: The ID of the user account.
- username: The username of the user.

```
{
  email: "johnDoe@gmail.com",
  firstName: "John",
  lastName: "Doe",
  role: 2,
  userId: 42,
  username: "newUser"
}
```

Listing 88. Example Response

POST api/users/

Creates a new user account. The new user must not have the same username or email as an existing user. The new user

will always start with the ”external user” role. Passwords are salted and hashed before being stored in the database. Any user can use this endpoint.

- email: The user’s email address.
- firstName: The user’s first name.
- lastName: The user’s last name.
- password: The user’s password.
- username: The user’s username.

```
{
  email: "johnDoe@gmail.com",
  firstName: "John",
  lastName: "Doe",
  password: "hunter2",
  username: "newUser"
}
```

Listing 89. Example Request Body

- insertId: The ID of the new user.

```
{
  insertId: 42
}
```

Listing 90. Example Response

POST api/users/search

Returns a limited list of users that match the search text and that only includes users that are at or beyond the cursor. The search text can find matches for first name, last name, username, email, or user ID. Only admins can use this endpoint.

- text: The partial or full text that is used to find a match. Partial matches are allowed. For example: searching for ”oh” would be able to return ”John123” since ”oh” is contained with in ”John123”.
- role: The role to filter results by (0 = any, 1 = external user, 2 = internal user, 3 = editor, 4 = admin).
- sort: The value to sort results by (0 = username, 1 = name, 2 = user ID, 3 = email, 4 = created, 5 = role).
- order: The order in which to display results (0 = descending, 1 = ascending).
- cursorPrimary: This cursor should contain the value that we are sorting by of the user we want to start our search at. This cursor is used to allow infinite scrolling, while only loading a small number of page results at a time. If this is a new search then this value should be null.
- cursorSecondary: This cursor should contain the user ID of the user you want to start your search at. This cursor is used in instances where the primary cursor value is ambiguous. If this is a new search then this value should be null.

```
{
  text: "oregonstate",
  role: 0,
```

```
  sort: 0,
  order: 1,
  cursorPrimary: "null",
  cursorSecondary: "null"
}
```

Listing 91. Example Request Body

- users: The users returned from the search.
  - created: The date that the user account was created.
  - createdUnix: The date that the user account was created. This value is represented as the number of seconds that have elapsed since the Unix epoch.
  - email: The user's email address.
  - firstName: The user's first name.
  - lastName: The user's last name.
  - role: The user's role (0 = not logged in, 1 = external user, 2 = internal user, 3 = editor, 4 = admin).
  - username: The user's username.
- nextCursor: An object that contains the information needed to continue getting the remaining search results.
  - cursorPrimary: This cursor should contain the value that we are sorting by of the user we that should appear next in the search results. If this is null, then there are no more users.
  - cursorSecondary: This cursor should contain the user ID of the user that should appear next in the search results. If this is null, then there are no more users.

```
{
  users: [
    {
      created: "2020-09-01T02:00:00.000Z",
      createdUnix: 1598925600,
      email: "testeditor@oregonstate.edu",
      firstName: "John",
      lastName: "Doe",
      role: 3,
      userId: 67,
      username: "testEditor"
    }
  ]
  nextCursor: {
    primary: null,
    secondary: null
  },
}
```

Listing 92. Example Response

PATCH api/users/:userId

Updates a user's information. Only the user being updated or an admins can use this endpoint. Only admins can change a user's role.

- userId: The ID of the user to update.
- email: The user's new email address.
- firstName: The user's new first name.
- lastName: The user's new last name.
- role: The user's new role which can only be set by admins (0 = not logged in, 1 = external user, 2 = internal user, 3 = editor, 4 = admin).
- username: The user's username.
- oldPassword: The user's current password.
- newPassword: The user's new password.

```
{
  email: "johnDoe@gmail.com",
  firstName: "John",
  lastName: "Doe",
  role: 3,
  username: "newUser",
  oldPassword: "hunter2",
  newPassword: "hunter3"
}
```

Listing 93. Example Request Body

- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 94. Example Response

POST api/users/:userId/newPassword

Generates a new random password for a user. This endpoint is ideal for when an admin wants to give a user access to an account that they forgot the password to. Only admins can use this endpoint.

- userId: The ID of user account to generate a new password for.
- password: The new randomly generated password.

```
{
  password: "9RBahxx47qiKcny"
}
```

Listing 95. Example Response

2.13 View Endpoints

GET api/views/page/:pageId

Returns all of the views available to the current user for the specific page. Any user can use this endpoint.

- pageId: The ID of the page to get the views for.
- views: The views that can be accessed on this page.
  - pageId: The page that this view belongs to.
  - public: Whether this view is viewable by all users (1) or only viewable by the current user (0).
  - userId: The user ID of the user who created this view.
  - viewId: The ID of the view.
  - viewName: The name of the view.
  - headers: The headers and their filter settings for the current view.
    - headerId: The ID of the current header.
    - filters: An array of filter icon IDs. Each value represents a specific filter that is disabled on the current header while in this view.

```
{
  views: [
    {
      pageId: 2,
      public: 1,
      userId: 51,
      viewId: 11,
      viewName: "Opportunities",
      headers: [
        {
          headerId: 1,
          filters: [1, 2, 3, 4, 5]
        }
      ]
    }
  ]
}
```

Listing 96. Example Response

POST api/views/page/:pageId

Creates a new view for the specific page. Any logged in user can use this endpoint.

- pageId: The ID of the page where the new view is created.
- publicView: Whether this view is viewable by all users (1) or only viewable by the current user (0).
- viewName: The name of the new view.
- headers: The headers and their filter settings for the current view.
  - headerId: The ID of the current header.

- filters: An array of filter icon IDs. Each value represents a specific filter that is disabled on the current header while in this view.

```
{
  publicView: 0,
  viewName: "No pros or cons",
  headers: [
    {
      headerId: 1,
      filters: [1, 2]
    }
  ]
}
```

Listing 97. Example Request Body

- insertId: The ID of the new view.

```
{
  insertId: 5
}
```

Listing 98. Example Response

DELETE api/views/:viewId

Deletes a view. Only the user who created the view or an admins can use this endpoint.

- viewId: The ID of the view to delete.
- affectedRows: The number of table rows affected by this action.

```
{
  affectedRows: 1
}
```

Listing 99. Example Response

3 Contact Information

If you have any questions about this API that are not answered in this document feel free to reach out to me by email at silverware13@gmail.com.